openAG Tooling Surface

Status: Tooling draft — design-only. openAG has zero code yet; AMS is frozen until parity. This document specifies the surface: the CLI, the MCP tool set, and the interfaces by which humans and agents drive the one B-ROUND engine, read the graph, and inspect gaps/risk. It is the companion to `docs/openAG/standard.md` (the data-model contract) and `docs/openAG/engine-architecture.md` (the engine).

Date: 2026-10-12.

---

1. Design principles

The surface obeys four rules so that nothing about the tooling can violate the standard or the engine's confluence guarantee:

1. The surface never writes the graph directly. Every CLI command and MCP tool is a thin driver that submits work to the engine; all writes go through symbolic `reconcile`/`gate` (engine §3). Tools propose and read; the engine decides and commits.
2. One engine, eight entry points. There is no "scan command" and no "validate command" as independent pipelines — every mutating command is an entry point into B-ROUND via one of B0–B7 (standard §1, engine §1).
3. Envelope-uniform. Every MCP tool returns `{ok, <payload> | error:{code, message, details?}}` with the shared error-code vocabulary (§6). CLI commands mirror this with exit codes.
4. Gap-honest by construction. Read tools surface typed gaps and the risk VIEW; they MUST NOT hide ungrounded promises or report plain attributes / reservoir items as gaps (standard §4.3, §7).

---

2. CLI surface (**openag**)

A single entry point `openag` with behavior-aligned subcommands. Mutating subcommands drive B-ROUND; read subcommands project the converged graph.

2.1 Round-driving commands (mutate via B-ROUND)

command	behavior	drives	notes
`openag needs <target>`	B0	opens the continuous ROUND (seed graph of OPEN promises).	human-attention is the budget.
`openag extract <target>`	B1	rounds to promise-fixpoint (top-down intent + bottom-up code-scan).	`scan` survives only as the code-scan fact source.
`openag author <entity> <attribute>`	B2	human promise-fill ceremony (`note → save → author → apply`).	reuses CAP-DF ceremony (engine §9).
`openag re-extract <target>`	B3	incremental, identity-preserving round (θ-crosswalk sticky bind).	`DecisionCache` memoizes.

command	behavior	drives	notes
`openag learn`	B6	feeds lessons back → `RuleTuner` → re-enters B-ROUND.	append-only journals.

2.2 Graph-reading commands (project the converged graph)

command	behavior	reads
`openag represent <slice>`	B4	materialize ModelSlice → projectors → artifacts (md/html/svg).
`openag generate <slice>`	B5	graph → code + parity evidence (oracle-in-loop).
`openag gate [<slice>]`	B7	run `B-ROUND.gate` standalone: L1 well-formedness + L2 authority.
`openag gaps [--axis AN] [--class ...]`	—	list typed gaps (the gap register).
`openag risk [--top N]`	—	the risk VIEW over the gap register, sorted by consequence.
`openag walk <walkspec>`	—	run a declarative WalkSpec (select→…→emit cards).
`openag export-graph [-f graphml|gpickle|json-graph|dot]`	—	NN-compilable `MultiDiGraph` export (`CON-N2`).
`openag show <entity>`	—	entity drill-down: promises, end-states, evidence, provenance.

2.3 Global flags

- `--budget <spec>` — override the widening budget for this invocation (govern reads it; overflow surfaces as `gap:budget-exhausted`).
- `--provider <name>` / `--policy <path>` — select the LLM routing policy (engine §7.5); default routes all neural work through a mock provider so nothing hits a paid API without opt-in.
- `--deterministic-now <iso8601>` — pin every timestamp for reproducible runs (mirrors `AMS_DETERMINISTIC_NOW`).
- `--json` — emit the MCP envelope shape (§6) instead of human text.

---

3. Gap and risk views

Gaps and risk are not separate stores — both are VIEWs over the single graph (standard §4.6, §7).

- **`openag gaps`** walks `A7` for every demanded promise whose status is `typed-gap` and groups by gap class and source class (existence / prohibition / parametric / VoI). Filters: `--axis`, `--class`, `--component`. Output is deterministic (sorted by `(axis, class, owner)`).
- **`openag risk`** is the same register re-projected: each gap's latent risk `P(won't ground) × consequence` is read from the VoI terms (`consequence` = value, `P` = confidence) and sorted descending. This is the exact VIEW gate L2 surfaces (engine §4.3, §5). No new data is computed — risk is intrinsic, never stored.

Both commands MUST exclude plain attributes and reservoir items (standard §4.3): those are legitimate terminal non-gaps.

---

4. Gate invocation

`openag gate` (and the `architect_gate` MCP tool, §5) evaluates the two gate layers (engine §5):

- L1 — symbolic well-formedness (hard). Schema + constraint predicates. Exit non-zero / `ok:false` on any L1 failure — a malformed graph MUST NOT publish.
- L2 — decision-authority go/no-go. The economic test `top−VoI < cost−floor` AND critical-path confidence ≥ acceptance. Judgment calls escalate to the human/neural oracle. Provisional roll-ups publish WITH gaps visible — gate reports gaps, it does not silently block on them.

Every gate run appends a `GateEvent` to `.architecture/gates.jsonl` (fail-soft), feeding B6 learning.

---

5. MCP tool set

MCP tools are the agent-facing mirror of the CLI, grouped by concern. Each returns the uniform envelope (§6). Tools drive the same `B−ROUND` engine — there is no second code path.

5.1 Round-driving tools

tool	behavior	role
`architect_needs`	B0	seed the continuous ROUND.
`architect_extract`	B1	drive extraction to promise-fixpoint.
`architect_promise_note`	B2	attach a promise note (ceremony step 1).
`architect_promise_save`	B2	WorkOrder + Job from a note (step 2).
`architect_promise_author`	B2	author promise content via provider (step 3; batchable).
`architect_proposal_apply`	B2	apply the authored proposal (step 4).
`architect_re_extract`	B3	incremental identity-preserving round.
`architect_learn`	B6	record a lesson; re-enter B-ROUND.

5.2 Graph-reading tools

tool	role
`architect_slice`	materialize a ModelSlice (compressed context broker).
`architect_represent`	B4 project a slice → artifacts.
`architect_generate`	B5 graph → code + parity evidence.
`architect_gate`	B7 two-layer gate (§4).
`architect_gaps`	the gap register VIEW.

tool	role
architect_risk	the risk VIEW (§3).
architect_walk	run a WalkSpec; return cards.
architect_export_graph	NN-compilable export (CON–N2).
architect_component_health	SI&L record + trend for one component.
architect_evaluate	system-wide health + freshness + sil_summary.

5.3 Provider and policy

LLM work is provider-agnostic behind the IF-LLMProvider interface (engine §7.5). The .llm projector-name suffix is contractual: architect_promise_author (and any write-back projector) MUST reject a projector name without the .llm suffix as INVALID_ARGUMENT. Routing, per-call cost caps, fallback, and bundling are policy-driven via .architecture/llm/policy.yaml.

---

6. Envelope and error vocabulary

Every MCP tool returns:

```
{ ok: true,  <payload> }
{ ok: false, error: { code, message, details? } }
```

Shared error codes (CLI maps these to exit codes):

code	meaning
INVALID_ARGUMENT	malformed input (incl. missing .llm suffix).
NOT_FOUND	repo / entity / job / slice missing (details.reason).
PRECONDITION_FAILED	state guard (e.g. job not queued, no provider).
SCHEMA_VIOLATION	payload fails schema / constraint predicate.
INTERNAL	unexpected failure (single-line; no traceback leak).

Read tools are fail-soft where they touch journals (gaps/risk/health): a missing or malformed journal yields an empty view, never a raised error.

---

7. Graph-export access

openag export-graph / architect_export_graph expose the Stage-1 MultiDiGraph export (standard §9, docs/graph-export.md) as the stable interface (IF-GraphExport) for downstream consumers (arch-agent, the N2 compiler/trainer). Formats: graphml (default), gpickle, json-graph, dot. --stats prints node/edge counts without writing. The export MUST round-trip losslessly, preserve parallel typed edges, and keep promise → evidence provenance walkable (so output provenance is itself NN-compilable supervision).

---

8. Mapping to AMS tooling (fork manifest)

The surface forks AMS's existing CLI + MCP surface (engine §9); nothing is rewritten from scratch:

openAG surface	AMS origin	disposition
`openag extract` / `architect_extract`	v3 7-stage pipeline + `architect_pipeline`	FORK → B1 fact source + seeding.
`openag walk` / `architect_walk`	walk 6-stage runner + 13 WalkSpecs	REUSE (executable operator prototype).
`openag represent` / `architect_represent`	lifecycle substrate + projectors	FORK → B4.
`architect_promise_*` ceremony	CAP-DF `note → save → author → apply`	REUSE → B2.
`openag gate` / `architect_gate`	`architect_gate`	FORK → two-layer gate.
`architect_slice` / `architect_evaluate` / `architect_component_health`	OCA MCP tools	REUSE.
(none)	legacy 10-stage `pipeline/` CLI	DROP — superseded by B-ROUND.

Net: the legacy linear-pipeline CLI is the only surface dropped; every other command/tool forks or reuses prior AMS effort (zero-loss crosswalk, standard §4 / entity-model §11).

---

9. Status and non-goals

- Design-only. No code exists. This is the surface a future implementation will satisfy; a TDD build plan is intentionally deferred to a later session.
- Synchronic scope. Trade-space / decision-management (a diachronic meta-ROUND) and full A6 lifecycle browsing are deferred — no CLI or MCP surface for them in gen-0 (engine §8).
- AMS frozen until parity. The `openag` binary ships only once the engine reaches functional parity with the AMS surface it forks.